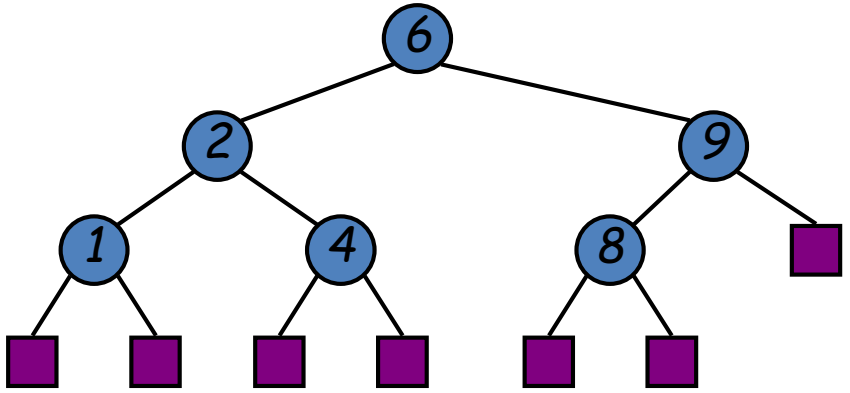
Binary Search Tree (BST)

- A binary search tree is a binary tree storing items (or key-data pair) at its internal nodes and satisfying the following property:
  - Let ***u***, ***v***, and ***w*** be three nodes such that ***u*** is in the left subtree of ***v*** , and ***w*** is in the right subtree of ***v***. We have
    key(u) $\leq$ ***key(v)*** $\leq$ ***key(w)***
- External nodes do not store items (for internal representation BST).

- An inorder traversal of a binary search trees visits the keys in increasing order

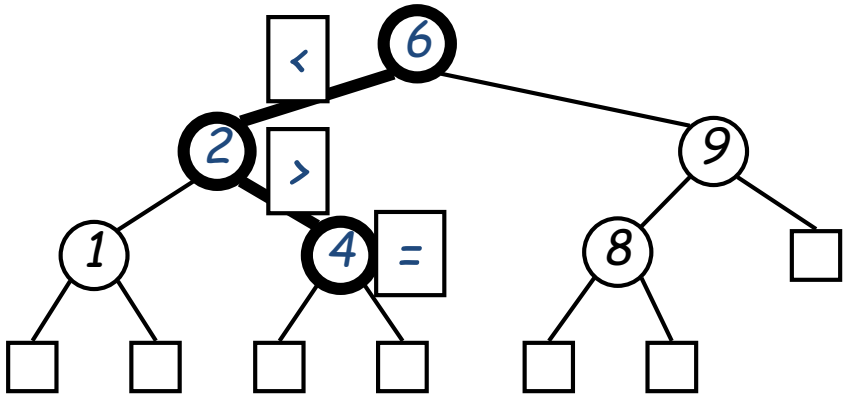


Search

- To search for a key ***k***, we trace a downward path starting at the root.
- The next node visited depends on the outcome of the comparison of ***k*** with the key of the current node.
- If we reach a leaf, the key is not found and we return a null position.

- Example: find(4)

```
Algorithm find (k, v)
    if T.isExternal (v)
        return Position(null)
    if k < key(v)
        return find(k, T.leftChild(v))
    else if k = key(v)
        return Position(v)
    else //k > key(v)
        return find(k, T.rightChild(v))
```



BST Operations: Delete

3 cases arise:

Case 1: x has no children:

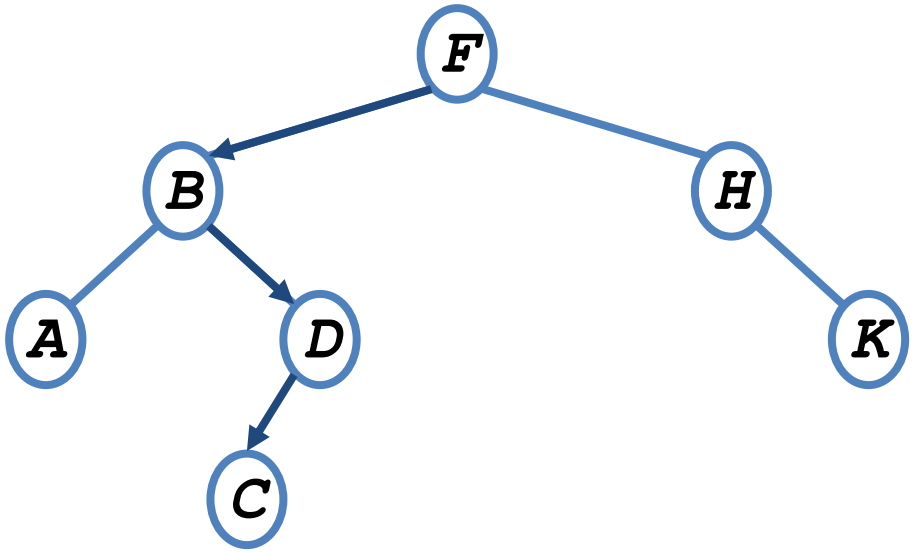
- Remove x

Case 2: x has one child:

- Splice x

Case 3: x has two children:

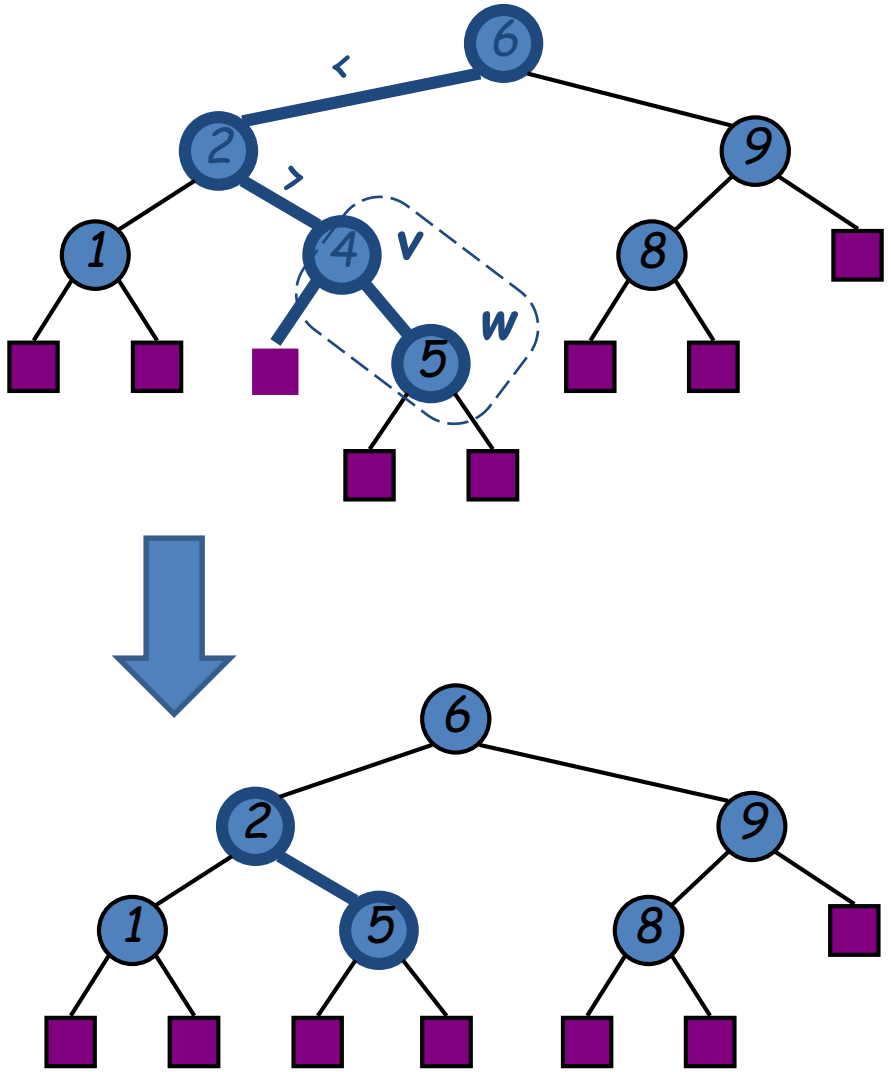
- Swap x with successor
- Perform case 1 or 2 to delete it



BST Operations: Delete (case 2)

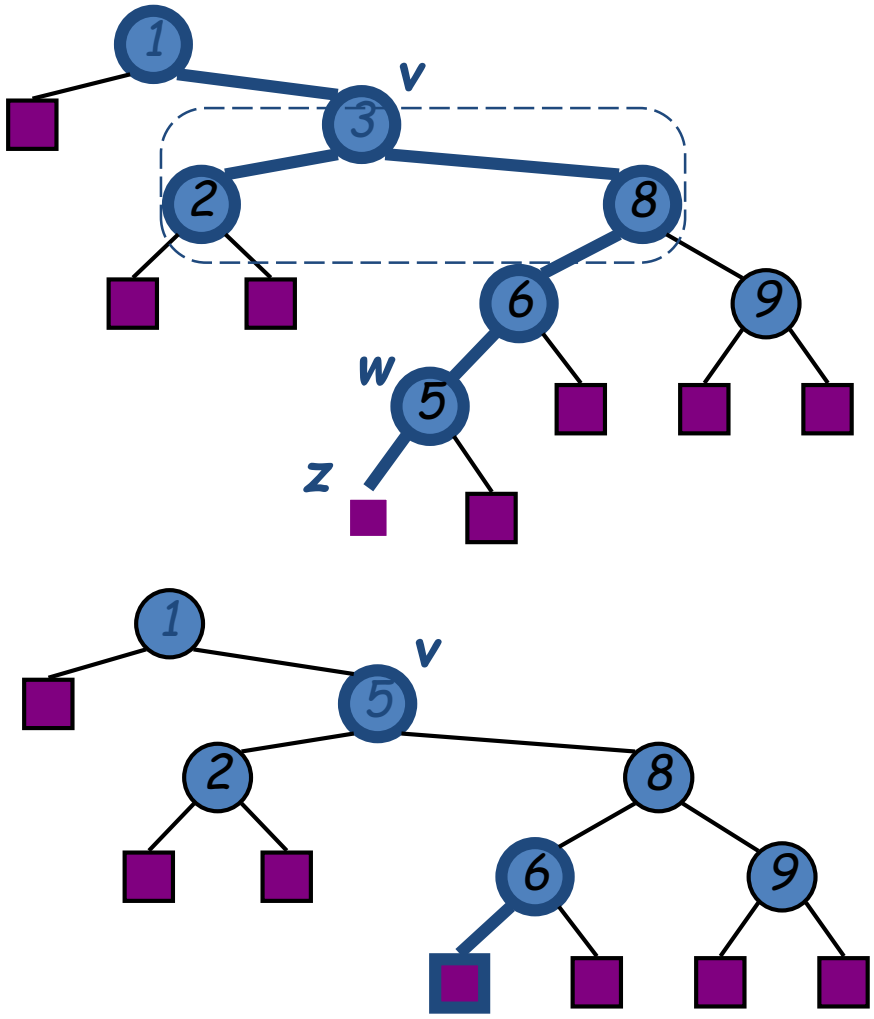
- To perform operation removeElement(***k***), we search for the key ***k***
- Assume the key ***k*** is in the tree, and let ***v*** be the node storing ***k***
- If node ***v*** has a child ***w***, we swap ***v*** and ***w*** , and remove v.

- Example: remove 4



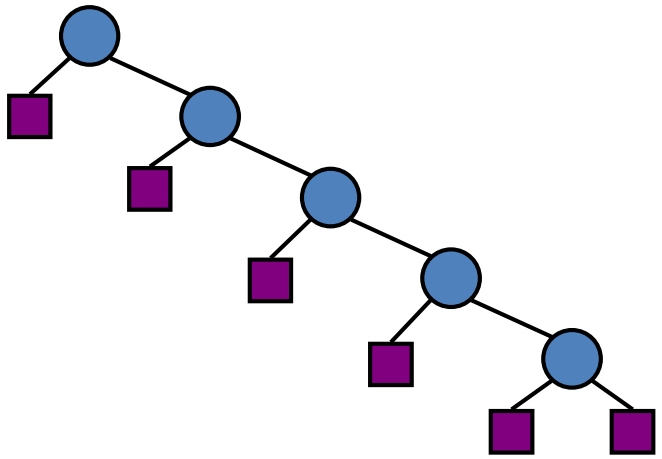
BST Operations: Delete (case 3)

- We consider the case where the key ***k*** to be removed is stored at a node ***v*** whose children are both internal
  - we find the internal node ***w*** that follows ***v*** in an inorder traversal
  - we swap ***v*** and ***w***
  - we remove node ***v***
- Example: remove 3

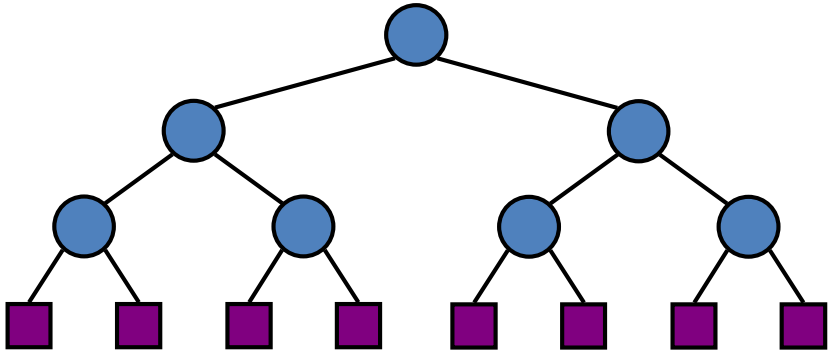


Performance

- Consider a dictionary with ***n*** items implemented by means of a binary search tree of height ***h***
  - the space used is ***O***(***n***)
  - methods findElement() , insertItem() and removeElement() take ***O***(***h***) time



- The height ***h*** is ***O***(***n***) in the worst case and ***O***(log ***n***) in the best case.



BST Operations: Insert

- To perform operation insertItem(k, o), we search for the key k.
- Assume k is not already in the tree, and let w be the leaf reached by the search.
- We insert k at node w and make w as an internal node.

- Example: insert 5

